



**Universidad  
Europea**

# PROYECTO INTEGRADOR

Técnico Superior en Desarrollo de Aplicaciones Web  
Daniel Correa Villa

Tutores:

Sara Villanueva Rosa  
Irene del Ricón Bello  
Raquel Cerdá Losa

## Introducción

El proyecto surge como una iniciativa dentro del ámbito académico para desarrollar una aplicación web destinada a gestionar las actividades y la información de un campamento infantil.

Con el objetivo de facilitar la administración y consulta de datos, este proyecto integra conocimientos de **Diseño de Interfaces, Desarrollo en Entorno Cliente, Desarrollo en Entorno Servidor y Despliegue de Aplicaciones Web.**

En su esencia, la aplicación busca ofrecer a los usuarios una plataforma intuitiva y funcional para manejar diversas actividades relacionadas con la organización del campamento. Desde la gestión de inscripciones hasta la comunicación con tutores legales y monitores, "Campamento Infantil" tiene como objetivo optimizar los procesos administrativos y mejorar la experiencia de los usuarios.

Con una estructura bien definida, una arquitectura Modelo Vista Controlador y un enfoque en la usabilidad, este proyecto busca mejorar la gestión y el disfrute de las actividades en un entorno educativo y recreativo.

Para ver el repositorio de la aplicación, acceda al siguiente link:

<https://github.com/DCV05/Proyecto-Integrador>

## Palabras clave

- Gestión de campamentos
- Validaciones en tiempo real
- Modelo Vista Controlador
- Diseño responsive
- Experiencia de usuario (UX)
- HTML
- CSS
- JavaScript
- JQuery
- Docker
- Git
- GitHub
- Polaris
- PHP
- Tailwind

## Índice

|                                                        |           |
|--------------------------------------------------------|-----------|
| <b><i>Introducción</i></b> .....                       | <b>2</b>  |
| Palabras clave.....                                    | 3         |
| Módulos formativos incluidos .....                     | 5         |
| Herramientas y lenguajes.....                          | 7         |
| <b><i>Objetivos</i></b> .....                          | <b>9</b>  |
| Objetivos generales .....                              | 9         |
| Objetivos específicos del proyecto .....               | 9         |
| <b><i>Desarrollo e implementación</i></b> .....        | <b>10</b> |
| Diagrama de casos de uso.....                          | 10        |
| Diagrama de entidad relación.....                      | 11        |
| Diagrama de modelo relacional.....                     | 12        |
| <b><i>Anexos</i></b> .....                             | <b>14</b> |
| Configuración general de Apache para el proyecto ..... | 14        |
| Configuración de autenticación Digest: .....           | 15        |

## Módulos formativos incluidos

- **Diseño de Interfaces:** Aplicación de técnicas de diseño centradas en el usuario (UX), basadas en las leyes de usabilidad aprendidas. Para la creación de prototipos y maquetas de las interfaces, se utiliza Figma, asegurando un diseño funcional y coherente desde la planificación hasta su implementación.
- **Desarrollo en Entorno Cliente:** Uso de jQuery y sus métodos para realizar validaciones en tiempo real, garantizando una interacción fluida con los formularios. Estas validaciones incluyen comprobaciones inmediatas al interactuar con los campos y mensajes de error dinámicos que se presentan junto a los elementos afectados.
- **Desarrollo en Entorno Servidor:** Implementación de la lógica de la aplicación utilizando PHP y conexión a bases de datos MySQL mediante la extensión **mysqli**. Esto permite gestionar de forma eficiente las operaciones del sistema y asegurar la correcta interacción entre el servidor y la base de datos.
- **Despliegue de Aplicaciones Web:** Configuración y gestión del servidor Apache utilizando las configuraciones aprendidas en clase, incluyendo el uso de archivos **.htaccess** para la gestión de rutas y control de accesos. Además, se emplea Docker para crear un entorno de desarrollo, optimizando la compatibilidad y el despliegue del proyecto al servidor de producción.
- **Entornos de desarrollo:** Uso de tecnologías como Git y GitHub para mantener un historial de cambios coherente y un repositorio con backups con el código de la aplicación en el caso de que ocurra algún tipo de error que borre el código o los recursos utilizados dentro del sistema.
- **Lenguaje de marcas:** Uso de HTML y CSS para el desarrollo de la estructura y estética de la aplicación. Implementación de JavaScript para programar el comportamiento de la página.

- **Bases de datos:** Desarrollo de bases de datos relacionales (MySQL) para guardar los datos de la aplicación.
- **Programación:** Implementación de la lógica, estructura de ficheros y programación orientada a objetos inspirada en Java. Conexión con MySQL y prepared statements.
- **Sistemas informáticos:** Uso de Linux como sistema operativo principal del proyecto. Instalación de MySQL y Apache para su uso en la aplicación. Navegación de directorios y modificación de ficheros mediante **Bash**.

## Herramientas y lenguajes

- **HTML:** Usado para definir la estructura de la página web y aplicar atributos que cumplen con los estándares de accesibilidad.
- **CSS:** Usado para aplicar estilos dentro de la aplicación y hacerla portable para los diferentes dispositivos.
- **Tailwind:** Aplicado para acelerar el desarrollo del diseño de interfaces, permitiendo implementar prototipos rápidamente y mantener un estilo visual consistente a lo largo del proyecto.
- **JavaScript:** Empleado para programar el comportamiento de la aplicación por parte del cliente y añadir distintos efectos visuales.
- **JQuery:** JQuery se ha utilizado específicamente para realizar validaciones en tiempo real en los formularios y gestionar eventos con el servidor.
- **PHP:** Usado como lenguaje principal de backend del proyecto. Por otra parte, PHP se encarga de la gestión de la lógica de la aplicación y conexión con MySQL.

**Polaris:** Polaris es un entorno de desarrollo basado en PHP y MySQL que facilita el desarrollo de proyectos. Este entorno gestiona las rutas mediante clases y cuenta con un compilador de etiquetas, permitiendo:

- Ejecutar métodos PHP para generar contenido dinámico.
  - Gestionar peticiones AJAX en una misma clase.
  - Proporcionar soporte multidioma basado en el idioma del navegador sin necesidad de configurar Apache o tener diferentes HTML.
- **MySQL:** MySQL ha servido como sistema de gestión de bases de datos para almacenar y organizar toda la información del proyecto.

- **PHPMyAdmin:** Utilizado para administrar las bases de datos, realizar consultas y gestionar tablas de una forma más sencilla.
- **Figma:** Figma ha sido clave para crear prototipos interactivos de las diferentes páginas de la aplicación y generación del kit de componentes usados.
- **Figjam:** Utilizado en las etapas iniciales del proyecto para diseñar las interfaces y planificar los objetivos y requisitos mínimos de cada entrega.
- **Docker:** Empleado como entorno de desarrollo local para garantizar que el proyecto funcione de manera uniforme en diferentes sistemas, encapsulando configuraciones y dependencias del servidor.
- **Git:** Usados para gestionar el control de versiones, tener backups, y mantener un historial de los cambios realizados durante las diferentes entregas.
- **GitHub:** GitHub ha proporcionado una plataforma para almacenar el proyecto.
- **Visual Studio Code:** Visual Studio Code ha sido empleado como editor de código principal debido a su amplia variedad de extensiones con las que se puede, por ejemplo, conectar Git y Docker.
- **Intellisense:** Usado para autocompletar tipado de PHP y gestionar errores dentro del código de una forma más visual.



## Objetivos

### Objetivos generales

El objetivo principal del proyecto es desarrollar una aplicación web moderna y funcional que permita gestionar todos los aspectos operativos de un campamento infantil, desde la inscripción de participantes hasta la organización de actividades y la comunicación con los tutores legales.

Por otro lado, se debe permitir gestionar los pagos de los diferentes participantes dentro del campamento.

### Objetivos específicos del proyecto

- Diseñar una interfaz de usuario intuitiva y accesible que cumpla con las buenas prácticas de UX/UI aprendidas en **Diseño de Interfaces Web**.
- Implementar formularios con validaciones en tiempo real para asegurar la correcta captura de datos.
- Crear secciones públicas y privadas que gestionen el acceso a las funcionalidades según el tipo de usuario.
- Establecer una estructura dinámica para mostrar y modificar información en tiempo real mediante la manipulación del DOM.
- Optimizar la comunicación con el servidor utilizando peticiones asincrónicas con JSON.

## **Desarrollo e implementación**

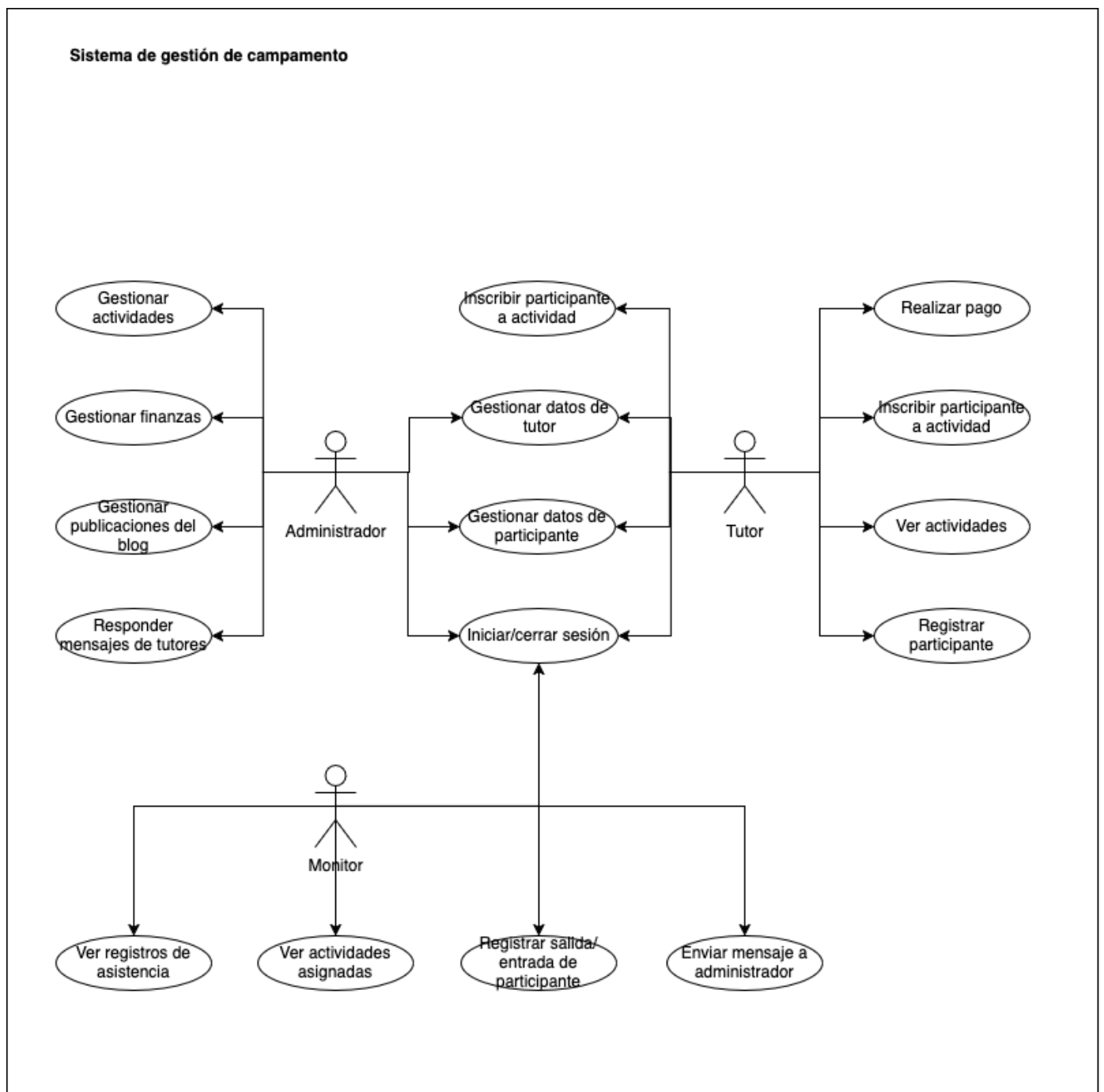
Para facilitar el desarrollo de la aplicación, se han creado varios diagramas:

### **Diagrama de casos de uso**

El diagrama de casos de uso ha sido desarrollado para visualizar las diferentes acciones que puede realizar cada actor según su rol dentro de la aplicación, así como la forma en que cada rol interactúa con el sistema.

## Diagrama de entidad relación

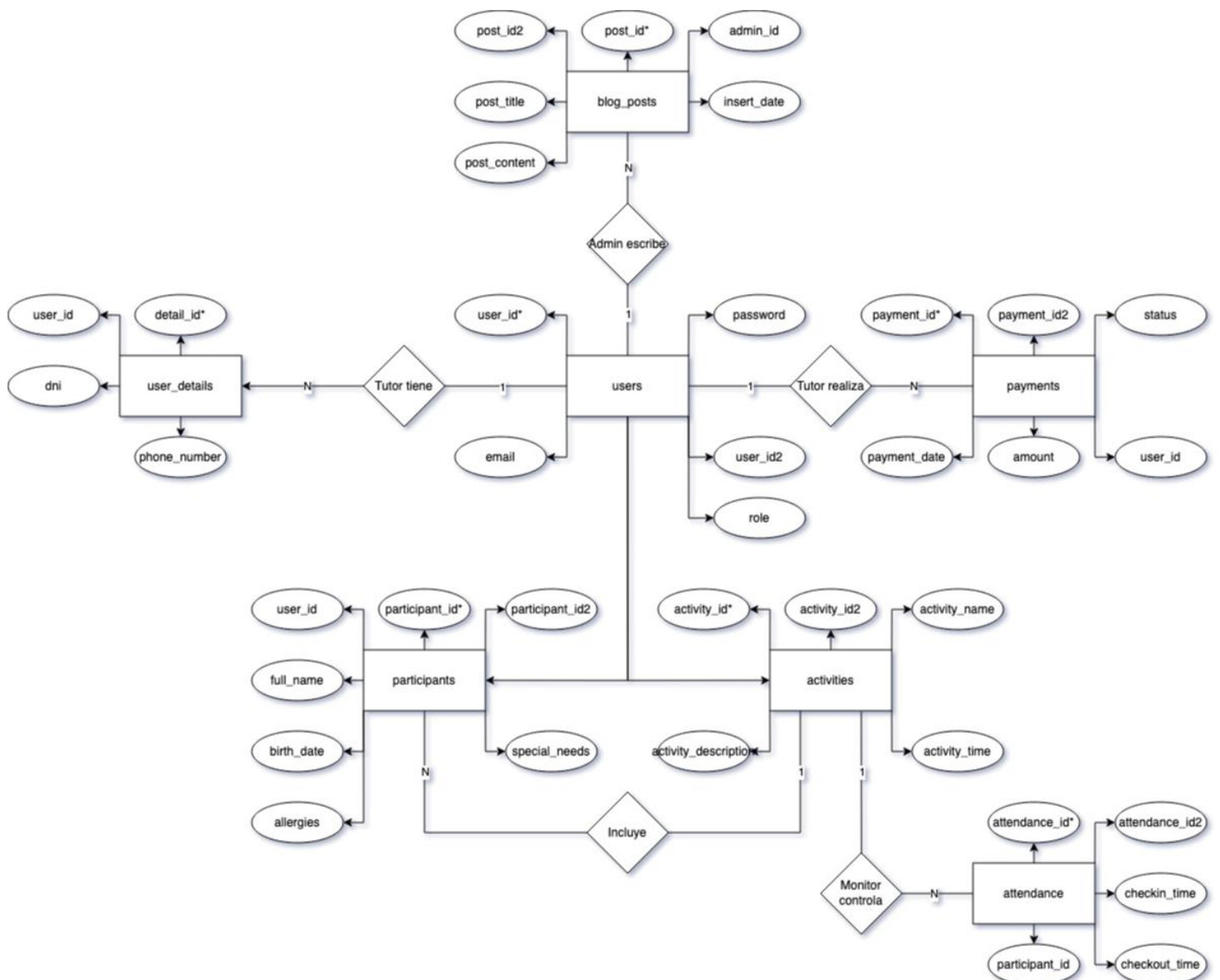
El **diagrama entidad-relación** ha sido desarrollado para representar las entidades principales del sistema, sus atributos y las relaciones entre ellas. Este modelo permite

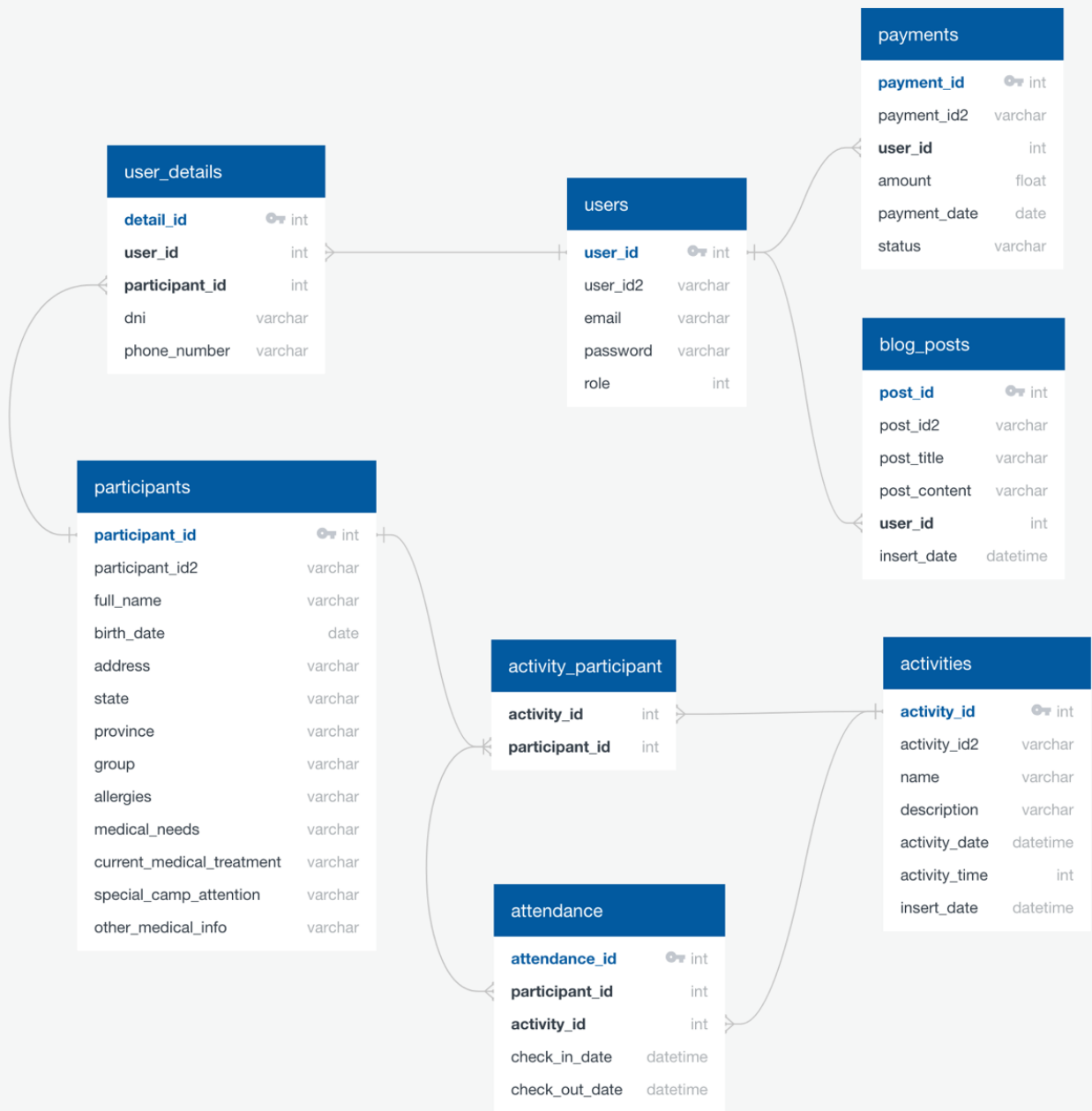


visualizar cómo los datos están estructurados y cómo las entidades interactúan entre sí, facilitando la organización y gestión de la información en la base de datos.

## Diagrama de modelo relacional

El **modelo relacional** ha sido desarrollado para definir las tablas de la base de datos, sus columnas, tipos de datos y relaciones mediante claves primarias y foráneas.





## Anexos

### Configuración general de Apache para el proyecto

Para desplegar correctamente el proyecto en el servidor de producción es necesario realizar numerosas configuraciones en diferentes servicios. Entre los más importantes se encuentra Apache, ya que es el servidor web que será utilizado durante el desarrollo de la aplicación.

La configuración general de Apache (la del propio servidor) no ha sido realmente modificada, ya que, en el caso de este proyecto, han sido usados archivos .htaccess. Por otra parte, han sido insertados varios .htaccess para manejar la autenticación de los usuarios en apartados privados (monitores y tutores) dentro del sistema.

En el caso de este proyecto, la configuración general de Apache en .htaccess ha sido la siguiente:

```
src > .htaccess
1  # No mostramos los ficheros
2  Options -Indexes
3
4  # Determinamos que el fichero principal es el index.php
5  DirectoryIndex index.php
6
7  # Configuramos que siempre que entremos en esta carpeta, siempre se ejecute el index.php
8  # Además, en el caso de que se intenten capturar archivos físicos o directorios, estarán disponibles
9  RewriteEngine On
10 RewriteCond %{REQUEST_FILENAME} !-f
11 RewriteCond %{REQUEST_FILENAME} !-d
12 RewriteRule ^.*$ index.php [L,QSA]
13
14 # Página personalizada para errores 404
15 ErrorDocument 404 /apache/errors/404.html
16
17 # Página personalizada para errores 403
18 ErrorDocument 403 /apache/errors/403.html
19
20 # Página personalizada para errores 500
21 ErrorDocument 500 /apache/errors/500.html
```

**Options -Indexes:** En el caso de que no exista el archivo DirectoryIndex, no se mostrarán todos los directorios del proyecto.

**DirectoryIndex:** Dentro del funcionamiento del proyecto, es necesario utilizar un archivo `index.php` para manejar todas las rutas y el propio funcionamiento del entorno, por lo que, para que funcione correctamente, es necesario hacer que este archivo se ejecute siempre.

**RewriteConditions:** Estas directrices indican que se debe de ejecutar siempre el `index.php` y que, en el caso de que se quiera acceder a algún directorio o archivo físico (por ejemplo, un archivo CSS), este sea accesible.

En caso de que no estuvieran estas directrices, al intentar enlazar un CSS o un JS a un archivo HTML, debe de ser procesado primero por el `index.php`

**ErrorDocument:** Estas directrices indican que si un error 404, 403 o 500, se deben de imprimir unos HTMLs específicos en pantalla.

## Configuración de autenticación Digest:

En los apartados de Monitor y Tutor, han sido creados dos archivos `.htaccess` para limitar el acceso a aquellos que tengan unas credenciales.

Para hacer esto, es necesario crear un archivo con las credenciales de los usuarios con el siguiente comando:

```
root@0a2fa606305e:/var/www/html# htdigest -c apache/.htdigest_monitors "Secured Area - Monitors" monitor_1
Adding password for monitor_1 in realm Secured Area - Monitors.
New password:
Re-type new password:
root@0a2fa606305e:/var/www/html# htdigest -c apache/.htdigest_tutors "Secured Area - Tutors" tutor_1
Adding password for tutor_1 in realm Secured Area - Tutors.
New password:
Re-type new password:
root@0a2fa606305e:/var/www/html#
```

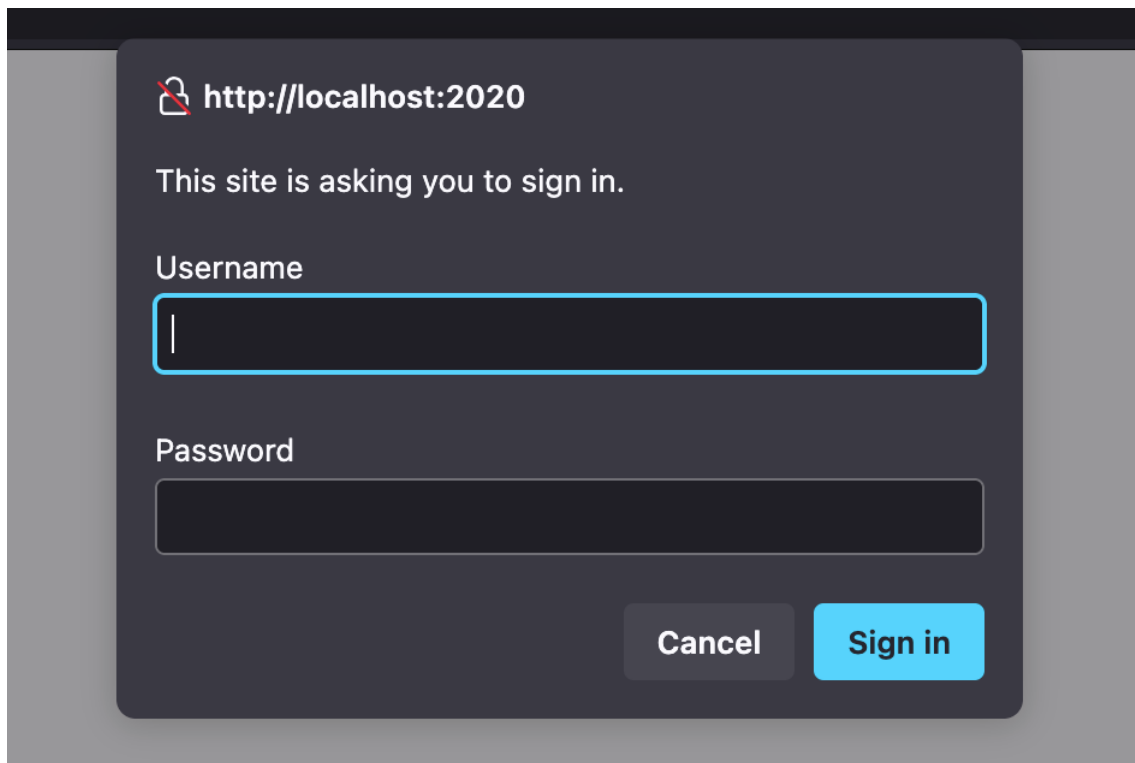
El resultado de este comando es que se crea un fichero `.htdigest` con las credenciales hasheadas.

```
src > apache > .htdigest_monitors
1 monitor_1:Secured Area - Monitors:184120b0ed29d296e9927023912c18b5
2 |
```

Después de ejecutar este comando, es necesario crear un archivo .htaccess en el directorio que queremos bloquear.

```
src > pages > TutorEdit > .htaccess
1  AuthType Digest
2  AuthName "Secured Area - Tutors"
3  AuthDigestProvider file
4  AuthUserFile /var/www/html/apache/.htdigest_tutors
5  Require valid-user
```

Una vez insertada esta configuración, al abrir la página donde esté este .htaccess, nos pedirá insertar unas credenciales.



The screenshot shows a web browser window with a dark-themed sign-in dialog. At the top, it displays the URL <http://localhost:2020> with a lock icon. Below the URL, it says "This site is asking you to sign in." There are two input fields: "Username" and "Password". The "Username" field is currently empty and has a blue border. At the bottom right, there are two buttons: "Cancel" and "Sign in".

Una vez hechas todas estas configuraciones, ya se habrá configurado correctamente la autenticación por Digest.